

Machine Learning 1

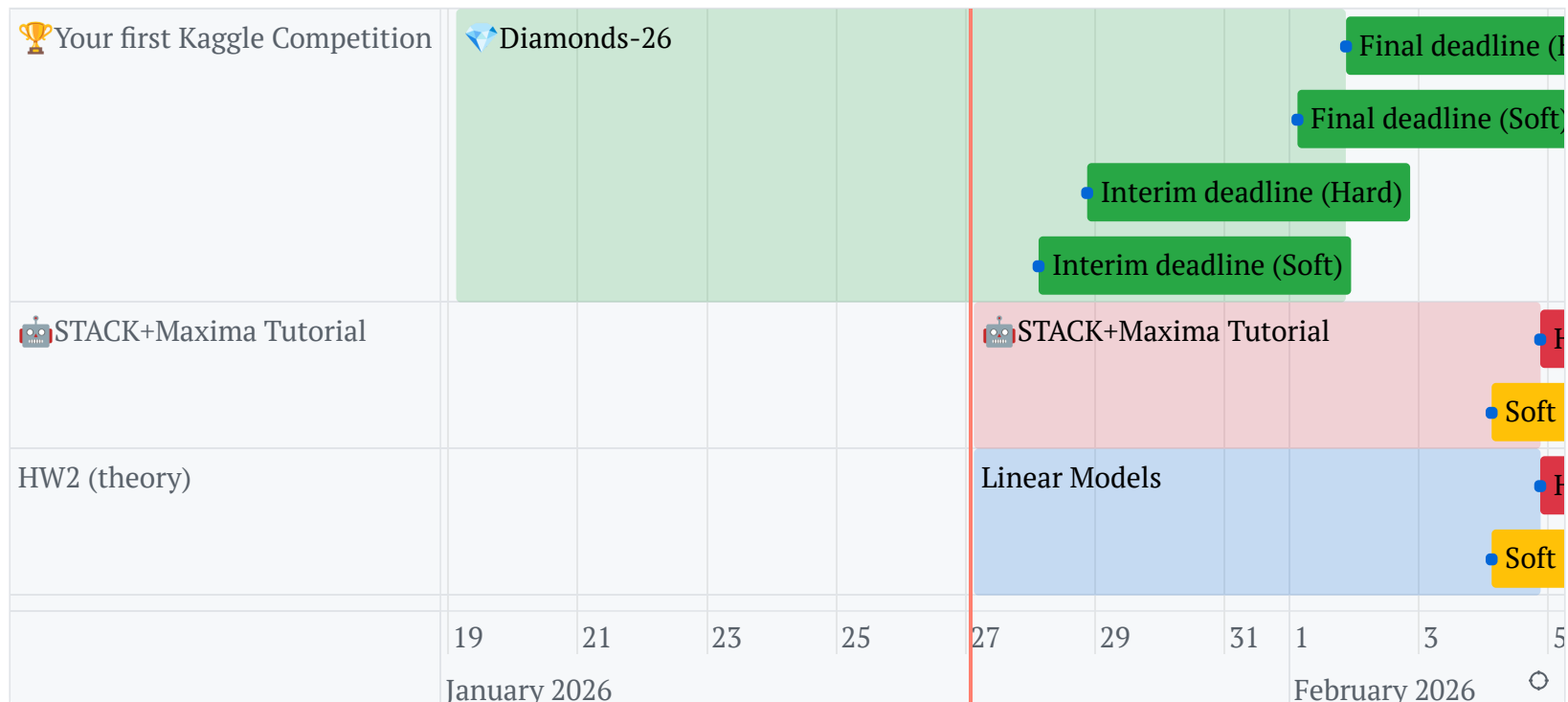
Classification

Alexey Boldyrev

27/01/2026

Image credit: 'The Mayflower at Sea'
by Granville Perkins, 1876
Wallach Division Picture Collection
The New York Public Library.

Current Homework Assignments



Map of Estimators (models) in Sklearn

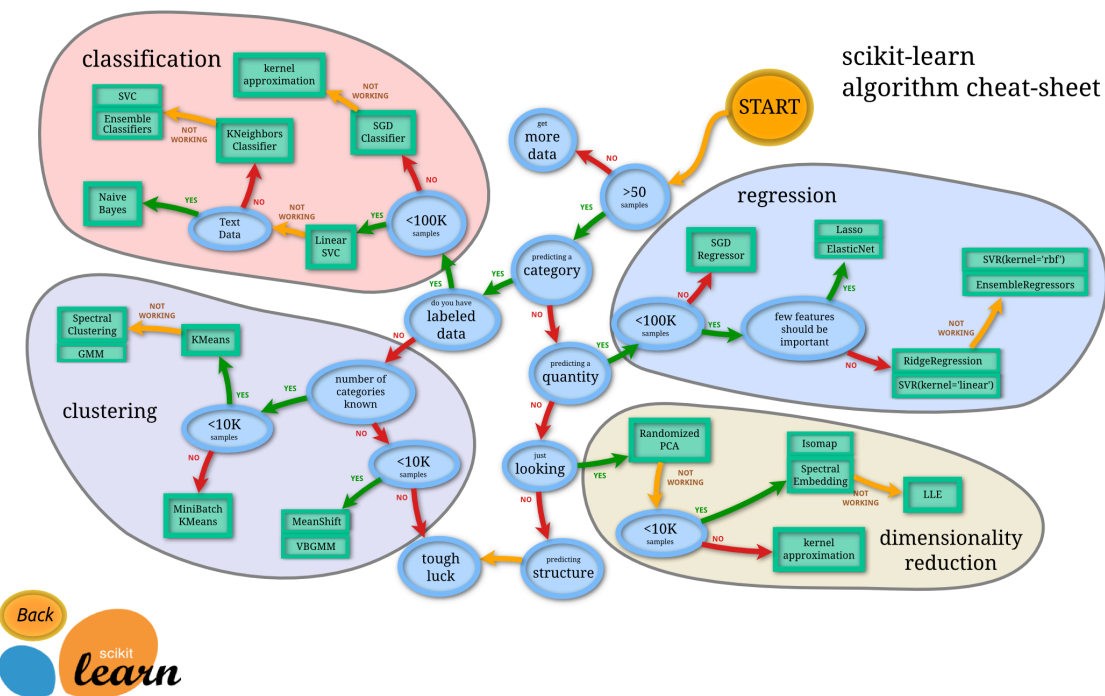


Image source: https://scikit-learn.org/1.4/tutorial/machine_learning_map/index.html

- **Supervised:**
 - Regression
 - **Classification**
- **Unsupervised:**
 - Clustering
 - Dimensionality reduction
- **Other:**
 - Reinforcement learning
 - Semi-supervised

Classification

- Linear regression handles **quantitative** response
- What if response is **qualitative**: male/female, cat/dog/rat/ant, bad/neutral/good?
 - These are **classes, categories, levels, factors**
 - The appropriate models in supervised-learning are **classifiers**
- Examples of classifiers:
 - **Logistic regression** ← Today's lecture
 - **Linear Discriminant Analysis (LDA)** ← Today's lecture
 - **Quadratic Discriminant Analysis (QDA)** ← Today's lecture
 - Suitable for both regression and classification:
 - **Classification And Regression Tree (CART)** ← Future lectures
 - **K-Nearest Neighbor (KNN)** ← Future lectures
 - **Support Vector Machine (SVM) and Classifier (SVC)** ← Future lectures

Classifier Comparison

- “No free lunch” (NFL) Theorem by David Wolpert: “Any two optimization algorithms are equivalent when their performance is averaged across all possible problems”
 - In a world of uncertainty, “All models are wrong, but some are useful”, George Box
 - Linear models: great for linear decision boundaries, but underperform in non-linear
 - Neural networks are general purpose models, but can underperform too

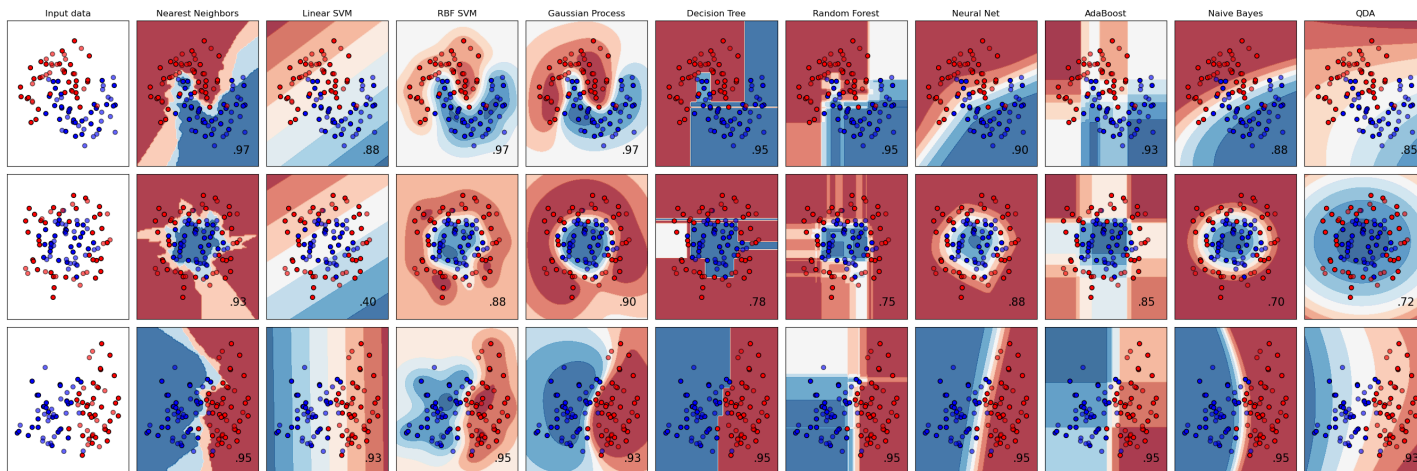
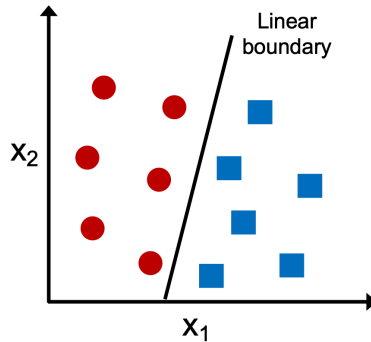


Image source: https://scikit-learn.org/stable/auto_examples/classification/plot_classifier_comparison.html

Note 1: Linear Separability

Linearly separable

A linear decision boundary that separates the two classes exists



Not linearly separable

No linear decision boundary that separates the two classes perfectly exists

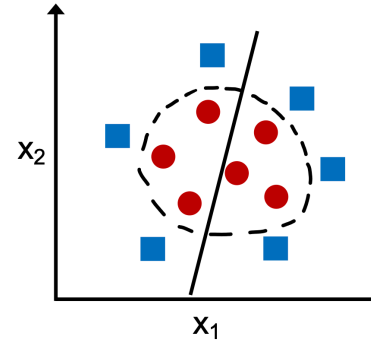
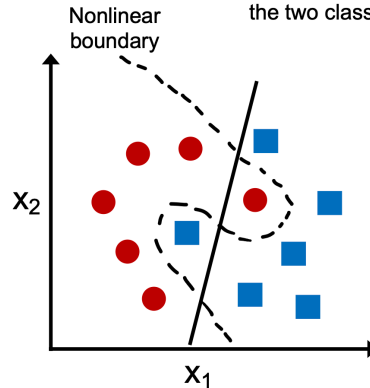


Image source: <https://github.com/rasbt/machine-learning-book/>

Note 2: Under/Overfitting in Classification

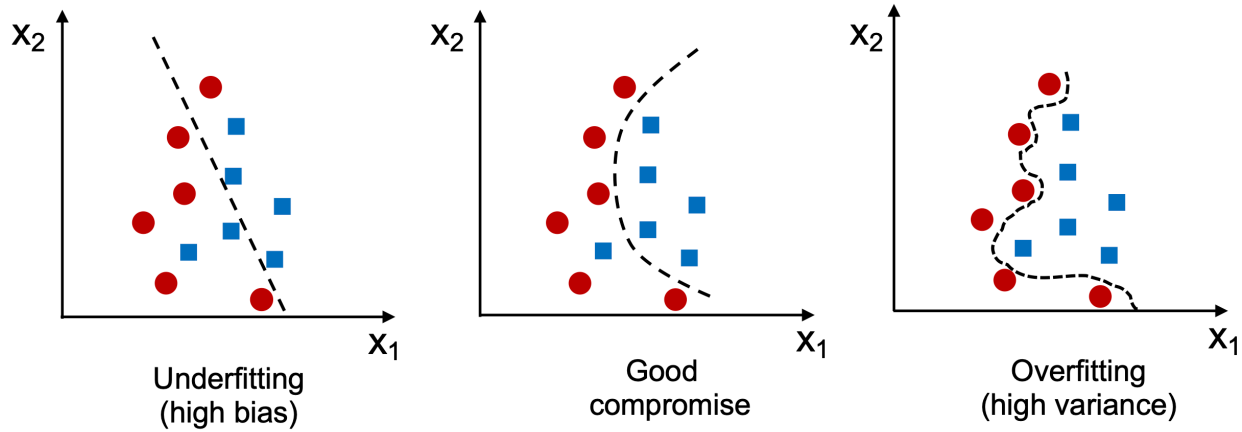


Image source: <https://github.com/rasbt/machine-learning-book/>

Eg. Credit Card Debt *Default* Dataset

- **Goal:** classify individuals into **default** and **non-default** categories of Y
- **Features:** Income, Balance (owed to credit card issuer), Student (Yes/No)
- **Response levels:** **default**, **non-default**

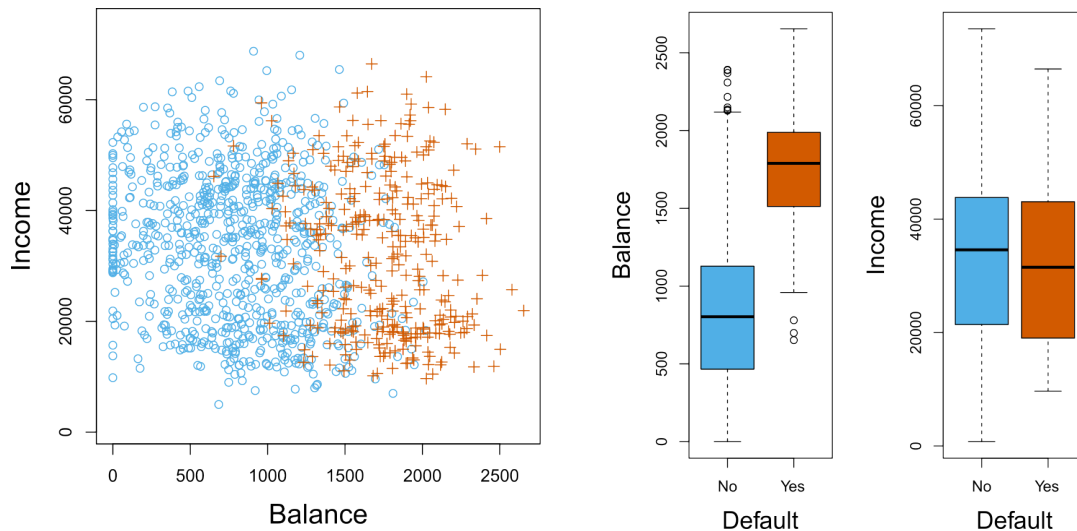


Image source: ISLP Fig. 4.1

Logistic Regression

Can We Classify with Linear Regression?

- Yes, but interpretation and model fit are suboptimal
- **Logistic regression** uses **logistic** function: "better" fit of extremes b/c of its S shape
 - We scale Y to $[0, 1]$ interval and interpret it as probability
 - We can **threshold** the probability to derive class 0 (negative) or class 1 (positive)
 - We compute $p(\text{balance}) := \mathbb{P}[\text{default} = \text{Yes} | \text{balance}]$
- Linear regression cannot be interpreted as a probability

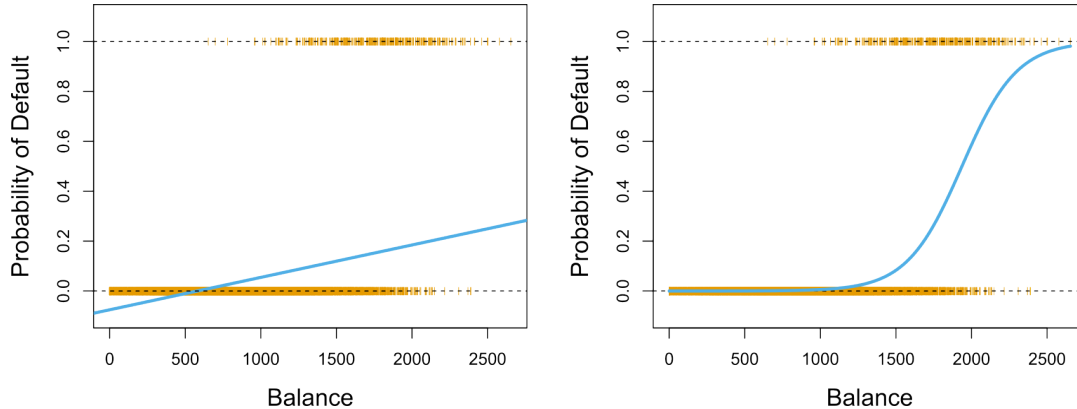


Image source: ISLP Fig. 4.2

Response Variable's Encodings

- Some models can't use **Cat** and **Dog** levels & require numeric encodings. Why?
 - Mathematical operations require numbers; strings cannot be multiplied by coefficients
- **Binary** response:
 - We can encode **{Cat, Dog}** with **dummy variables** as **{0, 1}** or **{1, 0}**
 - The model fit and predictions do not change, but we predict the probability of class **1**
- **Ternary** (with 3 levels) or higher response:
 - **One-hot** vectors (or dummy variables) avoid imposing order or metric on **Y**
 - **{Cat, Dog, Rat}** can be encoded as **{[0, 0], [0, 1], [1, 0]}** or similar combination
 - This does not imply any distance or ordering between levels
 - **Ordinal** encodings are better suited for ordinal variables (but this imposes metric)
 - **{Cold, Cool, Warm, Hot}** can be encoded with **{0, 1, 2, 3}**
 - This tells the model to treat distances between levels as **Hot - Warm $\approx$ Warm - Cool ...**
 - However, equal spacing may not always hold in practice

Logistic Model

- We want to model the probability $p(X) := \mathbb{P}[Y = 1|X]$ as a linear function
 - $p(X) = \beta_0 + \beta_1 X + \epsilon$ is linear regression with aforementioned problems

- A suitable model is via **logistic** function:

$$p(X) = \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}}$$

- We can derive **odds ratio** as $\frac{p(X)}{1-p(X)} = e^{\beta_0 + \beta_1 X}$

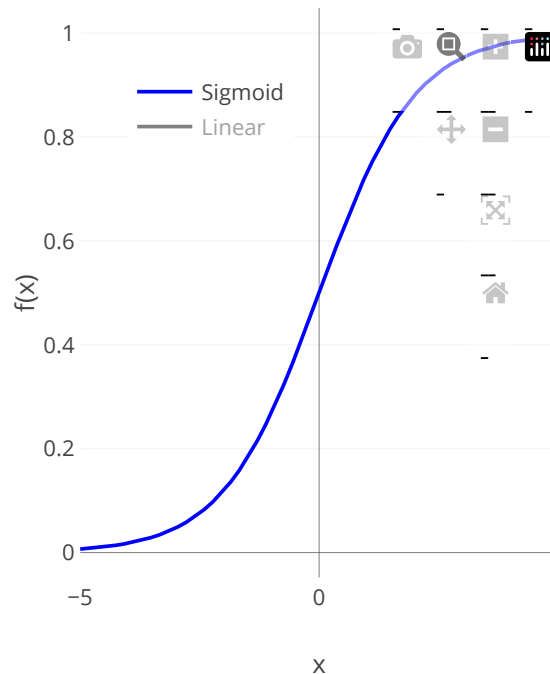
- We can further construct a **log-odds** (or logit function):

$$\log \left[\frac{p(X)}{1-p(X)} \right] = \beta_0 + \beta_1 X + \epsilon$$

- It is linear function **of** X , linear **in** betas
- Essentially, we are building a linear regression for the log-odds response

Sigmoid Function

- Sigmoid (logistic) function: $f(x) := \frac{1}{1+e^{-x}}$
 - Differentiable on $\mathbb{R}$ (just compute a derivative)
 $\implies$ continuous on $\mathbb{R}$
 - FYI: continuity does not imply differentiability
 - $f(-\infty) = 0, f(0) = 0.5, f(\infty) = 1$
 - Poorly distinguishes values "far from zero", but almost **linear** near zero
 - $f'(x)|_0 = f(x)(1 - f(x))|_0 = \frac{1}{2} \cdot (1 - \frac{1}{2}) = \frac{1}{4}$
is a rate of change of f at zero
 - $f''(x)|_0 = f(x)(1 - f(x))(1 - 2f(x))|_0 = \frac{1}{4} \cdot 0 = 0$ is a rate of change of derivative at zero



- Sigmoid is a CDF with the corresponding bell-shaped derivative, i.e. PDF

Estimation of Coefficients

- As always, we want the coefficients that are most likely to produce the observed values
 - under the given null hypothesis of model family, hyperparameters, etc.

- A likelihood function:

$$\ell(\beta_0, \beta_1 | x) \stackrel{\text{i.i.d. } X_i}{=} \prod_{i: y_i=1} p(x_i) \cdot \prod_{j: y_j=0} (1 - p(x_j))$$

- $i : y_i = 1$ are indices of all positive cases (a.k.a. class 1)
 - $j : y_j = 0$ are indices of all negative cases (a.k.a. class 0)
 - Recall: $p(x_i) = \mathbb{P}[Y|X = x_i]$ and we fixed it as
- Then we maximize the likelihood over the arguments:

$$\hat{\beta}_0, \hat{\beta}_1 := \arg \max_{\beta_0, \beta_1} \ell(\beta_0, \beta_1)$$

- Because of numerical underflow, we use log likelihood $\log \ell()$

Ex. Card Balance

```
import math
a = 0.5 # threshold probability
b, s = 2000, 1 # balance, student (yes|no)
e = math.exp(-10.6513 + 0.0055 * b) # p.142 ISLP
p = e / (1 + e) # conditional probability of default
print(f'P[default = Yes | balance=${b}] = {p:.3f}')
print(f'P[default = No | balance=${b}] = {1 - p:.3f}')
print(f'Decision at {a} = {"" if p>a else "not"} likely to default')

P[default = Yes | credit card balance=$2000] = 0.586
P[default = No | credit card balance=$2000] = 0.414
Decision at 0.5 = likely to default
```

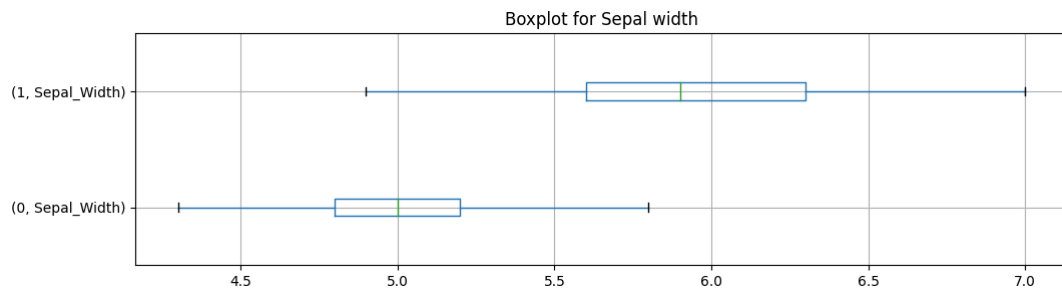
Ex. Predicting Default From Student Status

```
import math
a = 0.5 # threshold probability
b, s = 2000, 1 # balance, student (yes|no)
e = lambda x=1: math.exp(-3.5041 + 0.4049 * x)
p = lambda x=1: e(x) / (1 + e(x)) # conditional probability of default
print(f'P[default | student] = {p(s):.3f}')
print(f'P[non-default | student] = {1 - p(s):.3f}')
print(f'P[default | non student] = {p(1-s):.3f}')
print(f'P[non-default | non student] = {1 - p(1-s):.3f}')
print(f'Decision for a student at {a} = {"" if p(s)>a else "not"} likely to default')
print(f'Decision for a non-student at {a} = {"" if p(1-s)>a else "not"} likely to default')

P[default | student] = 0.043
P[non-default | student] = 0.957
P[default | non student] = 0.029
P[non-default | non student] = 0.971
Decision for a student at 0.5 = not likely to default
Decision for a non-student at 0.5 = not likely to default
```

Making Predictions (i.e. Inferences)

- Let's explore famous Iris data set
 - Goal: classify iris species based on petal and sepal widths and lengths
- 3 classes (species), but let's use just two
 - 50 observations per class
- 4 features, but let's just use sepal with
 - How well does this feature separate classes?
 - Consider sepal width of $x_0 \in \{4, 5, 5.5\}$



Images source:
https://en.wikipedia.org/wiki/Iris_flower_data_set

Ex. Fisher's Iris Data Set

```
import pandas as pd, numpy as np; from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
# Iris types: 0 = Setosa, 1 = Versicolour, 2 = Virginica
# Features (in cm): sepal length/width, petal length/width
X, y = load_iris(return_X_y=True)
X, y = X[y!=2], y[y!=2] # drop Virginica iris
X = X[:,[0]]
vX = X[49:50, :] # observations to classify

df = pd.DataFrame(X, columns=['Sepal_Width']);
ax = df.groupby(y).boxplot(vert=False, subplots=False, figsize=(12,3));
tmp = ax.set_title('Boxplot for Sepal width');

clf = LogisticRegression(random_state=1, penalty='none', fit_intercept=True)
clf.fit(X, y) # fit coefficients to training observations (gradient descent business)
betas = np.array(list(clf.intercept_) + list(clf.coef_[0]))
LogOdds = clf.intercept_ + vX * clf.coef_[0] # linear combination = log-odds
Odds = np.exp(LogOdds) # exp of log-odds = odds
pX = Odds/(1+Odds) # convert odds to probability
alpha=0.5
```

Multiple Logistic Regression

- An extension to simple linear regression with log-odds:

$$\log \left(\frac{p(\mathbf{X})}{1 - p(\mathbf{X})} \right) = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p$$

- Probability of class 1 (positive) given $\mathbf{X} := \mathbf{X}_{1:p}$:

$$p(\mathbf{X}) = \frac{\exp[\beta_0 + \beta_1 \mathbf{X}_1 + \dots + \beta_p \mathbf{X}_p]}{1 + \exp[\beta_0 + \beta_1 \mathbf{X}_1 + \dots + \beta_p \mathbf{X}_p]}$$

Confounded (Correlated) Predictors

- Overall default rates are higher for students
- If balance is a predictor, then students have lower default rates
 - This is because students tend to have higher credit card balances and higher balances are associated with higher defaults

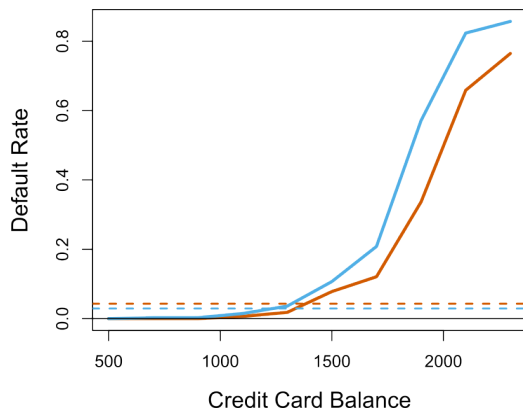
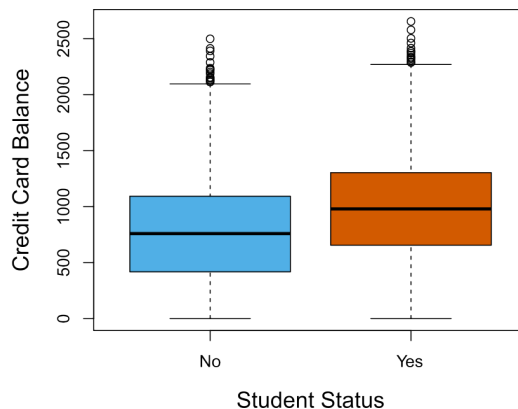


Image source: ISLP Fig. 4.3



Logistic Regression for >2 Response Classes

- Consider $Y \in \{\text{cat}, \text{dog}, \text{rat}\}$ with $K = 3$ classes
- Logistic regression handles binary response, i.e. $p(X)$ or $1 - p(X)$, but we happy to apply it to K classes in **one-versus-all** (OVA or one-vs-rest) fashion:

$$p_k(X) := \mathbb{P}[Y = k|X]$$

$$\log \frac{p_k(X)}{p_{-k}(X)} := \beta_{k0} + \beta_{k1}X = [1X]\beta_k$$

- we only need to do this $K - 1$ times, since $\sum_k p_k(X) = 1$
- So, we have K linear functions:
 - $X \rightarrow X'\beta_{\text{cat}}, X \rightarrow X'\beta_{\text{dog}}, X \rightarrow X'\beta_{\text{rat}}$
- **Multinomial logistic regression** (a.k.a. softmax regression) with **multilogit** function:
 - It uses a generalized logistic function, called **softmax**: $p_k(\mathbf{X}) := \frac{e^{\mathbf{X}'\beta_k}}{\sum_j e^{\mathbf{X}'\beta_j}}$
 - So, if $p_{\text{cat}}(\mathbf{X}) = \mathbf{0.2}$, $p_{\text{dog}}(\mathbf{X}) = \mathbf{0.3}$, then: $p_{\text{rat}}(\mathbf{X}) = \mathbf{1} - \mathbf{p}_{\text{cat}}(\mathbf{X}) - \mathbf{p}_{\text{dog}}(\mathbf{X}) = \mathbf{0.5}$

One-versus-one (OVO) approach

- We can also apply logistic regression for any pairs of K response levels
 - This yields $\frac{K(K-1)}{2}$ pairs of inputs
 - Advantage: if you start with equal number of observations in each class (say, $n_k = 10$), then each logistic regression deals with **balanced classes**
 - In **one-versus-all**, we will have 10 observations in class k and 20 observations in the rest

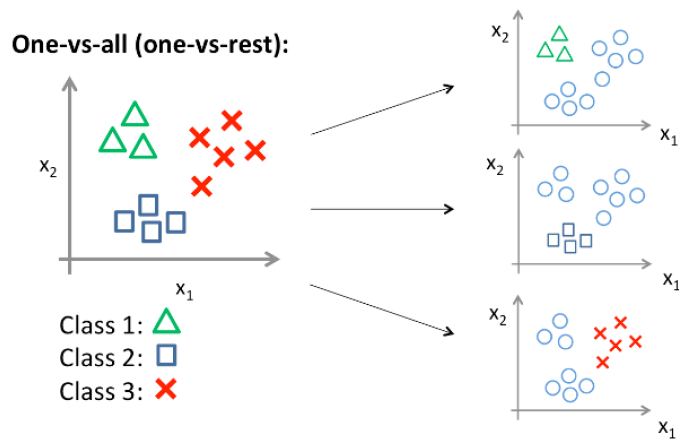


Image source: https://sites.cc.gatech.edu/classes/AY2016/cs4476_fall/results/proj4/html/jnanda3/index.html

Fitting Logistic Regression for 2 Classes

- We assume i.i.d. $X_i \sim \text{Binomial}(\theta_{2 \times 1} := (n, p))$
 - For $K > 2$, we assume i.i.d. $X_i \sim \text{Multinomial}(\theta_1, \dots, \theta_{K-1})$
- Then maximize log likelihood estimator (MLE) of θ :

$$\ell(\theta) = \log \text{Bin}(\mathbf{X}|\theta) = \log \prod \text{Bin}(\mathbf{X}_i|\theta) = \sum \log \text{Bin}(\mathbf{X}_i|\theta)$$

- In terms of $\beta := [\beta_0, \beta_1]$, we have:

$$\ell(\beta) = \sum_{i=1}^N \left\{ y_i \log p(x_i; \beta) + (1 - y_i) \log (1 - p(x_i; \beta)) \right\} = \sum_{i=1}^N \left\{ y_i \beta^T x_i - \log(1 + e^{\beta^T x_i}) \right\}$$

- Take derivative, set it to zero to find critical values (min, max, saddle), second derivatives to identify local max. Then determine global max
- See ESL textbook, p. 120-122

Linear Discriminant Analysis

Bayes Theorem

- 250+ years old formula (published posthumously in 1763)

$$\underbrace{f(\theta|x)}_{\text{posterior}} = \underbrace{f(\theta)}_{\text{prior}} \cdot \underbrace{f(x|\theta)}_{\text{likelihood}} \bigg/ \underbrace{f(x)}_{\text{marginal}} \propto \underbrace{f(\theta) \cdot f_{\theta}(x)}_{\text{unnormalized posterior}},$$

where the marginal density is a constant w.r.t. θ



Thomas Bayes (1702—1761)

- Goal:** infer the posterior distribution of $\theta|x$ from the assumed prior f and the likelihood f_{θ}
 - It combines **prior** knowledge about θ with the current **evidence** (data) to improve the posterior
 - Here x is fixed and θ and $\theta|x$ are RVs
- If we know the density $f(\theta) \cdot f_{\theta}(x)$, which must integrate to **1**, then:
 - The constant of integration, $f(x)$, can be determined numerically
 - Likewise, we can drop all constants (w.r.t. θ) in the likelihood and prior to simplify the expression

Effect of Observed Sample

$$\underbrace{f(\theta|x)}_{\text{posterior}} \propto \underbrace{f(\theta) \cdot f_{\theta}(x)}_{\text{prior} \cdot \text{likelihood}}$$

- A prior is our guess about the posterior
 - We can make predictions with no observation
 - In frequentist world, this is not possible
- With observed sample we update the posterior and improve predictions
- Posterior is a pointwise product

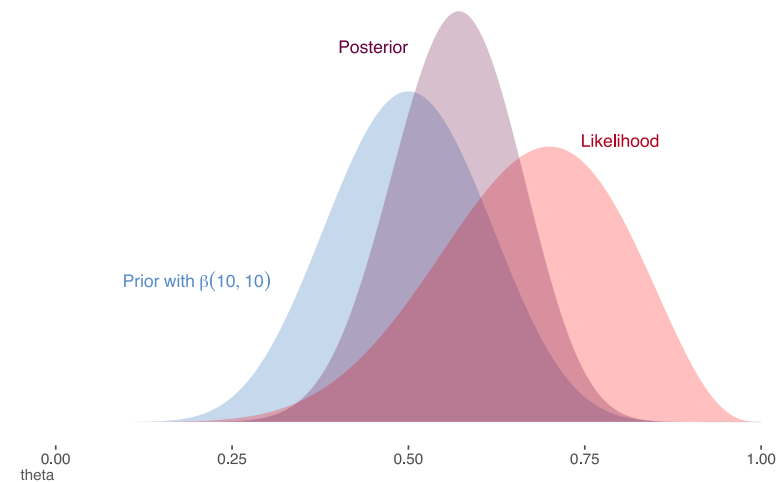


Image source: <https://m-clark.github.io/bayesian-basics/example.html>

Estimation of Posterior Distribution

- Goal: model **conditional distribution**, $Y|X$:

$$p_k(X) := \mathbb{P}[Y = k|X]$$

- Note: this probability is some function of X
- In logistic regression, we model it directly as $p_k(X) = \frac{e^{X'\beta}}{1+e^{X'\beta}}$, where X' is a transpose of X
- Alternative method:

$$\underbrace{p_k(X)}_{\text{posterior}} = \frac{\overbrace{\mathbb{P}[Y = k]}^{\pi_k, \text{prior}} \cdot \overbrace{\mathbb{P}[X|Y = k]}^{f_k(X)}}{\mathbb{P}X} = \frac{\pi_k \cdot f_k(X)}{\sum_i \pi_i f_i(X)}$$

- we estimate the class-specific distribution of X , $f_k(X)$
- Now, we have to estimate $\pi_k, f_k(X)$ for each class $k = 1 : K$
 - π_k is a **function of response value counts**, not the constant $\pi \approx 3.14159$

Estimating a Prior Probability Function, π_k

- Assuming a **random sample of observations** from a population
 - π_k - proportion of observations in class k
- Eg 1. Our sample has the following response values: {👹, 👹, 👹, 🕷️, 🕷️, 🕷️, 🕸️, 🕸️, 🕸️, 🕸️}

$$\vec{\pi} := [\pi_{\text{👹}}, \pi_{\text{🕷️}}, \pi_{\text{🕸️}}] = \left[\frac{3}{10}, \frac{3}{10}, 1 - \pi_{\text{👹}} - \pi_{\text{🕷️}} \right] = \left[\frac{3}{10}, \frac{3}{10}, \frac{4}{10} \right]$$

- Eg 2. We are trying to predict ad clicks with response label $Y \in \{\text{Yes}, \text{No}\}$ (user clicked ad or not). The training set has **1000** observations with clicks and **1M** without.

```
nYes, nNo = 1000, 1000000
print(f'prior for click = Yes class: {nYes/(nYes + nNo)}')
print(f'prior for click = No class: {nNo/(nYes + nNo)}')
prior for click = Yes class: 0.000999000999000999
prior for click = No class: 0.999000999000999
```

- Note: without any other information we expect **99.9%** of users to ignore ads

Estimating $f_k(X)$

- This one is tricky, unless we assume some distribution family
- In **Linear Discriminant Analysis (LDA)**, we assume $f_k \equiv N(\mu_k, \sigma_k^2)$ and need to estimate parameters of the Gaussian

$$f_k(x|\mu_k, \sigma_k) := \frac{1}{\sqrt{2\pi}\sigma_k} \exp \left[-\frac{(x-\mu_k)^2}{2\sigma_k^2} \right]$$

- Note: for K classes we assume K Gaussian distributions
- The assumption $\sigma = \sigma_k, \forall k$ simplifies our posterior as:
$$p_k(x) = \frac{\pi_k \cdot f_k(X)}{\sum_i \pi_i f_i(X)} = \frac{\pi_k \exp[-0.5\sigma^{-2}(x-\mu_k)^2]}{\sum_i \pi_i \exp[-0.5\sigma^{-2}(x-\mu_i)^2]}$$
- Try: take **log** of both sides and do some algebra to derive **discriminant function**

$$\delta_k(x|\mu_k, \sigma) := x \frac{\mu_k}{\sigma^2} - \frac{\mu_k^2}{2\sigma^2} = \log \pi_k$$

- The class with largest δ_k "wins" the observation x
 - i.e. δ_k determines the LDA class **decision boundaries** for classifying x

Ex. Bayes Decision Boundary with Gaussian $X|Y$

- Here we have $X|_{Y=1,2} \sim N(\mu_\gamma = \pm 1.25, \sigma^2 = 1)$
- The classification uncertainty arises from the overlap in densities
- Best decision is to classify x to the class k with the **highest posterior density**, $p_k(x)$
 - LDA boundary approximates (theoretical) **Bayes decision boundary**

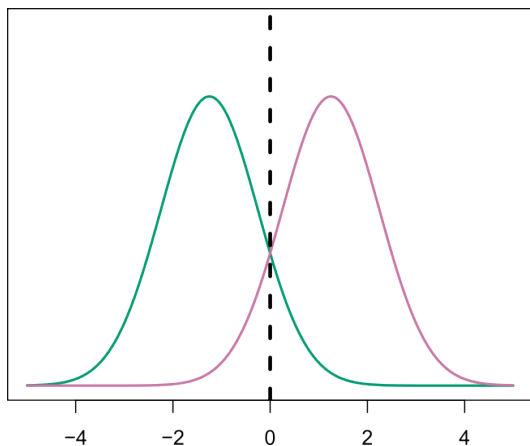
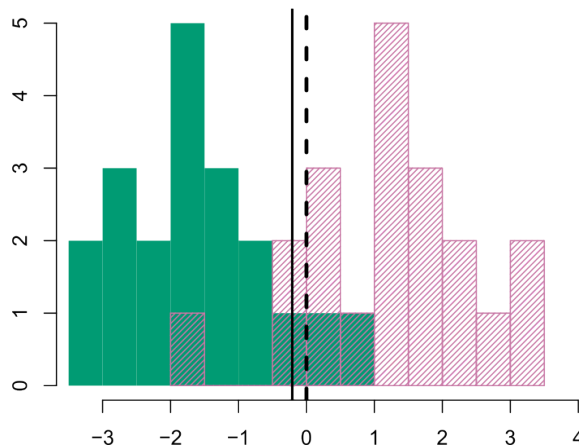


Image source: ISLP Fig. 4.4



Examples of non/overlapping densities $X|Y$

- Eg 1. Average pixel brightness is > 100 for cats and < 100 for dogs
 - $\text{supp}(X|\text{cat}) \cap \text{supp}(X|\text{dog}) = \emptyset$
- Eg 2. $X|Y = k \sim \text{Uniform}[k - 1, k], k = 1, 2, 3$
 - So, we know that X values in $[1, 2]$ could come from $k = 1$ only
 - What is the class of observation $X = 1.5$?
- Eg 3. However, $X|Y = k \sim \text{Uniform}[k - 1, k + 1], k = 1, 2$ have overlapping supports

Estimating Parameters in Discriminant Function, δ_k

- Assume $\sigma = \sigma_k, \forall k$
- Then we need to estimate $2K + 1$ params and plug them into δ_k to derive

$$\hat{\delta}_k(x) := x \cdot \underbrace{\frac{\hat{\mu}_k}{\hat{\sigma}^2}}_{c_1} + \underbrace{\frac{-\hat{\mu}_k^2}{2\hat{\sigma}^2} + \log \hat{\pi}_k}_{c_0} = c_0 + c_1 x$$

- $\hat{\delta}_k(x)$ is a linear function of x (i.e. linear in x)
- Parameters to estimate: $\mu_1, \dots, \mu_k, \pi_1, \dots, \pi_k, \sigma$
- $\hat{\mu}_k := \frac{1}{n_k} \sum_{i:y_i=k} x_i$, sample mean of x_i 's in class k
- $\hat{\sigma}^2 := \frac{1}{n-K} \sum_{k=1}^K \sum_{i:y_i=k} (x_i - \hat{\mu}_k)^2$, a sample variance of x_i 's in class k
- $n := \sum_k n_k$
- n_k - number of observations in class k
- $\hat{\pi}_k := \frac{n_k}{n}$, a prior probability of class k

LDA, $p > 1$

- We now assume RV $\mathbf{X}$ has a **multivariate normal distribution** in each class

$$\begin{bmatrix} X_1 \\ \vdots \\ X_p \end{bmatrix} | Y \sim N_p \left(\mu_Y := \begin{bmatrix} \mu_{1Y} \\ \vdots \\ \mu_{pY} \end{bmatrix}, \Sigma_Y := \begin{bmatrix} \hat{\sigma}_{11Y} & \dots & \hat{\sigma}_{12Y} \\ \vdots & \ddots & \vdots \\ \hat{\sigma}_{21Y} & \dots & \hat{\sigma}_{ppY} \end{bmatrix} \right), \text{ for each of } K \text{ classes}$$

- $\sigma_{ijY} := \text{Cov}(X_i, X_j | Y)$, $\sigma_{iiY} := \text{Cov}(X_i, X_i | Y)$, $Y = 1, \dots, K$

- We need to estimate parameters for K such distributions

- With true parameters we have:

- $\tilde{x} = x - \mu$

- $c := (2\pi)^{-\frac{p}{2}} |\Sigma|^{-\frac{1}{2}}$, $|\Sigma|$ determinant

- $f_k(x) := c \cdot \exp[-\frac{1}{2} \tilde{x}^T \Sigma^{-1} \tilde{x}]$

- $\delta_k(x) := x^T \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k + \log \pi_k$

- We classify x as $\arg \max_{\forall k} \delta_k(x)$

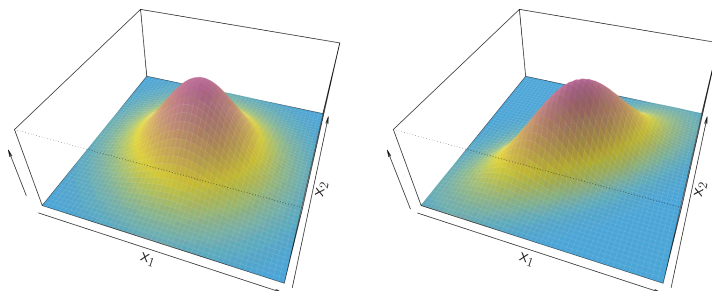
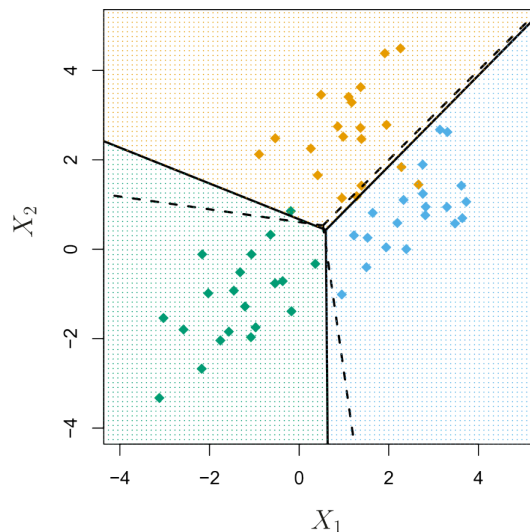
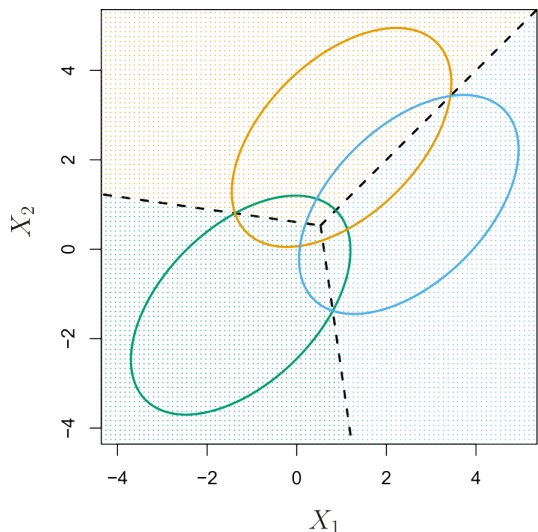


Image source: ISLP Fig. 4.5

Eg. $p = 2, k = 3$

- $Y \in \{GREEN, BLUE, ORANGE\}$
- $X|_{Y=GREEN} \sim N_2(\cdot, \cdot), X|_{Y=BLUE} \sim N_2(\cdot, \cdot), X|_{Y=ORANGE} \sim N_2(\cdot, \cdot)$
- The Bayes decision boundaries are $\{x | \delta_k(x) = \delta_\ell(x), k \neq \ell\}$
- LDA has linear decision boundaries and expresses $\delta_k(x)$ as a linear function of x



Classification Methods Demo

Demo of different classification methods

LDA: Key Takeaways

- **LDA** is a Bayesian classifier that models $P(X|Y = k)$ as Gaussian
- Assumes **shared covariance** Σ across all classes
 - This leads to **linear decision boundaries**
- Uses **Bayes' theorem** to compute posterior probabilities:
 - $P(Y = k|X) \propto P(Y = k) \cdot P(X|Y = k) = \pi_k \cdot f_k(X)$
- Classification: assign x to class k with highest discriminant $\delta_k(x)$
- **When to use LDA:**
 - Classes are approximately Gaussian with similar covariances
 - You have relatively few observations (lower variance than QDA)
 - You need interpretable linear decision boundaries

Classification Metrics

Why Not Just Use Accuracy?

- **Accuracy** = (Correct predictions) / (Total predictions)
- Problem: Accuracy can be misleading for **imbalanced datasets**!
- **Example**: Disease detection with 1% positive cases
 - A "classifier" that always predicts "No disease" achieves 99% accuracy!
 - But it's completely useless for detecting the disease
- **Solution**: We need metrics that capture class-specific performance
 - Confusion matrix, Precision, Recall, F1-score, ROC-AUC

Confusion Matrix

Consider 4 class classification problem:

		Expected class			
		1	2	3	4
Predicted class	1	52	3	7	2
	2	2	28	2	0
	3	5	2	25	12
	4	1	1	9	40

In the matrix:

- Value - number of objects of j -th class that were predicted as i -th class
- Diagonal elements are **true** classifications
- Off-diagonal elements are **false** classifications

Binary Classification

Consider a binary classification problem with classes + and -

		Expected class	
		+	-
Predicted class	+	TP (True Positive)	FP (False Positive)
	-	FN (False Negative)	TN (True Negative)

Binary Classification

Consider a binary classification problem with classes + and -

		Expected class	
		+	-
Predicted class	+	TP (True Positive)	FP (False Positive)
	-	FN (False Negative)	TN (True Negative)

Quality metrics:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

Binary Classification

Consider a binary classification problem with classes + and -

		Expected class	
		+	-
Predicted class	+	TP (True Positive)	FP (False Positive)
	-	FN (False Negative)	TN (True Negative)

Quality metrics:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

$$\text{True positive rate, TPR, Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Binary Classification

Consider a binary classification problem with classes + and -

		Expected class	
		+	-
Predicted class	+	TP (True Positive)	FP (False Positive)
	-	FN (False Negative)	TN (True Negative)

Quality metrics:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

$$\text{True positive rate, TPR, Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$\text{False positive rate, FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

Binary Classification

Consider a binary classification problem with classes + and -

		Expected class	
		+	-
Predicted class	+	TP (True Positive)	FP (False Positive)
	-	FN (False Negative)	TN (True Negative)

Quality metrics:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

$$\text{True positive rate, TPR, Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$\text{False positive rate, FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Binary Classification

Consider a binary classification problem with classes + and -

		Expected class	
		+	-
Predicted class	+	TP (True Positive)	FP (False Positive)
	-	FN (False Negative)	TN (True Negative)

Quality metrics:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

$$\text{True positive rate, TPR, Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

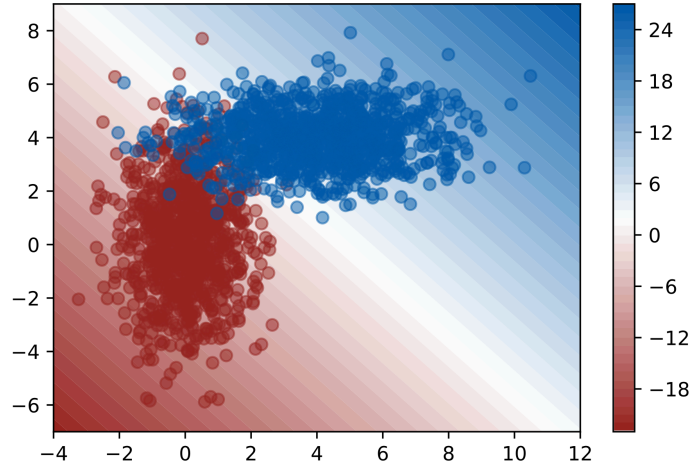
$$\text{False positive rate, FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

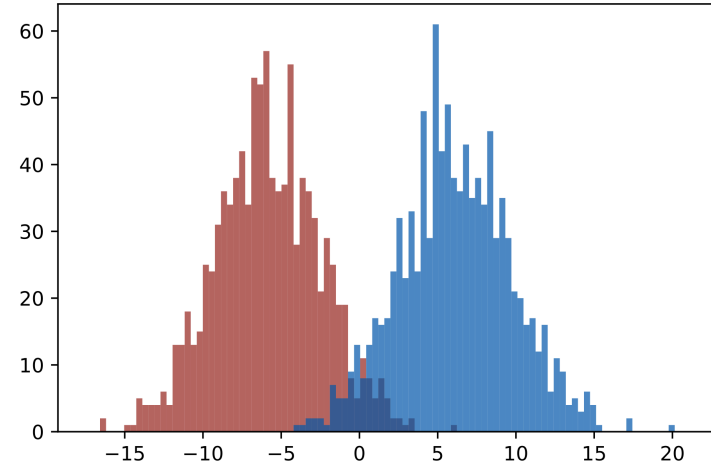
$$F_1\text{-score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Predicting Continuous Values

Many classification algorithms deal with continuous evaluation function



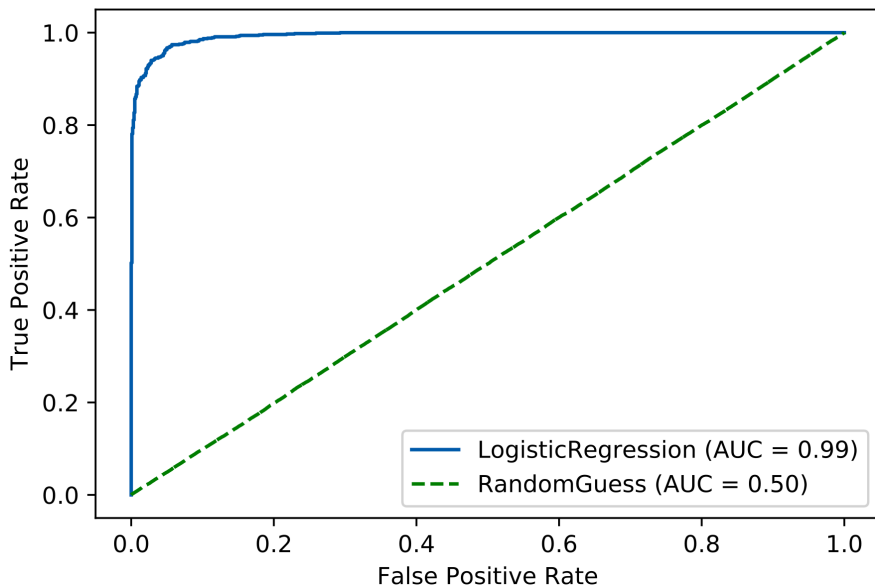
Prediction of the classification model



The value of the evaluation function in the data

ROC Curve

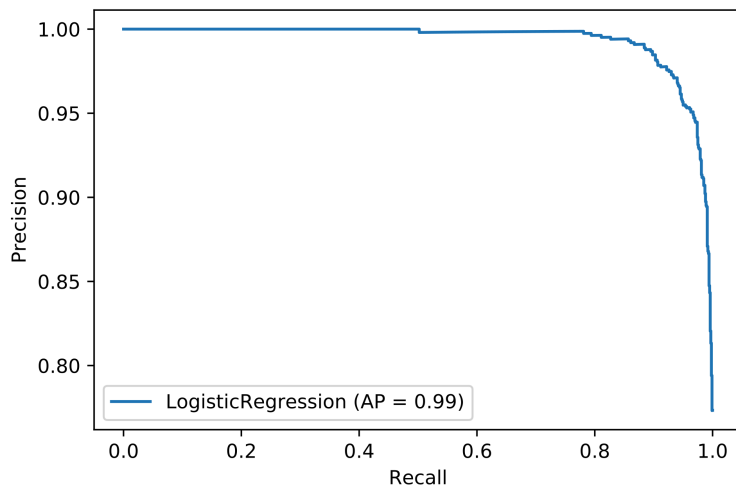
Receiver Operating Characteristic = **TPR** as a function of **FPR**



- All thresholds are used to generate ROC
- Useful for comparing different models without focusing on any threshold
- Area Under Curve (AUC) summarized the ROC into a single value
 - Model comparison can be automated
- $AUC \in [0, 1]$ with max value when ROC "hugs" top left corner
- If ROC follows "no information" line then predictors are unrelated to probability of interest

Interactive demo

Precision-Recall (PR) Curve



- The PR curve shows the trade-off between Precision and Recall for different thresholds
- A large area under the curve indicates both:
 - High Precision value (low false positive rate)
 - High Recall value (low false negative rate)

Confusion Matrix. Eg. Default Dataset

- **Diagonal**: correct predictions
- **Off-diagonal**: misclassified & needs attention
- **Accuracy rate** (score) is $(9644 + 81)/10000 = 97.25\%$ with error rate of **2.75%**
 - A **null classifier** tags everyone as non-defaulters produces $333/10000 = 3.33\%$ error rate.
Impressive?!?
 - Accuracy is a poor metric for **imbalanced classes**
 - Imbalanced because one class is much larger than another
- **"Positives"** = defaulters, the class of interest to us
- **"Negatives"** = the other class (non-defaulters)
- We need class-specific metrics of performance

		<i>True default status</i>		
		No	Yes	Total
<i>Predicted default status</i>	No	9644	252	9896
	Yes	23	81	104
Total		9667	333	10000

Source: ISLP Table, 4.4

Specificity & Sensitivity (Recall). Eg. Default Dataset

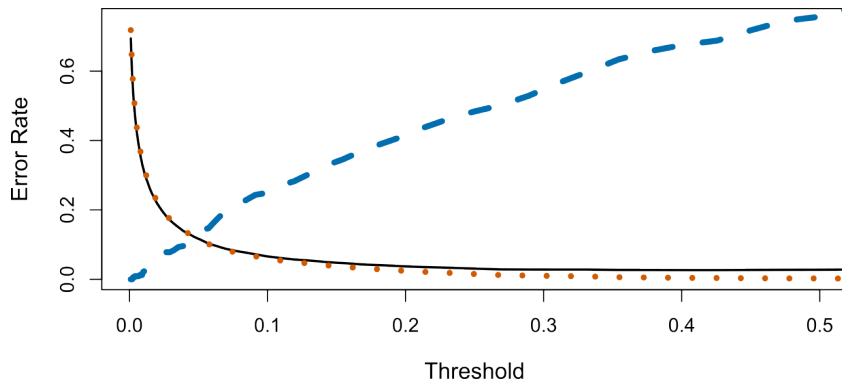
- **Specificity** or % of correctly identified true negatives
 - Out of 9667 actual non-defaults 9644 were correctly predicted, 99.8%
- **Sensitivity** (or recall) of % of correctly identified true positives
 - Out of 333 actual defaults 81 were correctly predicted, 24.3%
 - Recall is poor because Bayes LDA is maximizing accuracy
- A bank may have greater concern with defaulters
 - Defaulters are costlier than non-defaulters
 - Bank may want to identify more defaulters at a cost of misclassifying non-defaulters
- So, we can lower the **threshold** of identifying positives
- Eg. $\mathbb{P}[\text{defaulter} | X] > 0.2$ are classified as defaulters
 - Then, we predict 430 defaulters
 - includes 138 of true positives
 - recall $\uparrow$ 41.4% and accuracy $\downarrow$ 96.3%

		True default status		
		No	Yes	Total
Predicted default status	No	9644	252	9896
	Yes	23	81	104
Total		9667	333	10000

Source: ISLP Table, 4.4

Type I & II Error Rates

- False positive (FP): a **non-defaulter** labeled as **defaulter**
- False negative (FN): a **defaulter** is labeled as **non-defaulter**
 - $\text{FP rate} := \mathbb{P}[\text{Type I error}]$
 - $\text{FN rate} := \mathbb{P}[\text{Type II error}]$
 - $\text{TPR} = 1 - \text{FNR}$
- Lower threshold "flags" more **defaulters** (lower FNR) and **non-defaulters** (higher FPR)



Quadratic Discriminant Analysis

Quadratic Discriminant Analysis (QDA)

- LDA assumes $X|_Y \sim N(\mu_Y, \Sigma)$
- QDA assumes $X|_Y \sim N(\mu_Y, \Sigma_Y)$, where $\Sigma_Y \in \mathbb{R}^{p \times p}$ matrix, $Y = 1 : K$
- The QDA discriminant function is then
$$\delta_k(x) := -\frac{1}{2} \cdot \underbrace{\tilde{x}_k^T \Sigma_k^{-1} \tilde{x}_k}_{\text{quadratic } x \text{ term}} - \underbrace{\frac{1}{2} \log |\Sigma_k| + \log \pi_k}_{\text{constant term}}$$
- where $\tilde{x}_k := x - \mu_k$
- Compare this to LDA's:
$$\delta_k(x) := \underbrace{x^T \Sigma_k^{-1} \mu_k}_{\text{linear } x \text{ term}} - \underbrace{\frac{1}{2} \mu_k^T \Sigma_k^{-1} \mu_k + \log \pi_k}_{\text{constant term}}$$
 - Here we only compute a single inverse
- Classification is the same: the class k with highest $\hat{\delta}_k(x)$ wins x
- Estimation: Σ_k is symmetric requiring $\frac{1}{2}p(p+1)$ estimates k times
 - 50 predictors requires 1275 covariance estimates in $\Sigma_1, \dots, \Sigma_k$

LDA vs QDA

- LDA is less flexible and can only fit linear decision boundaries
- LDA avoids overfitting, tends to have lower variance
 - Works better for fewer observations
 - Computational performance is linear with the number of features
- QDA works better for larger n , when risk of overfitting is low
 - Note Σ_k does not increase with n (computing Σ_k^{-1} is $\mathcal{O}(n^3)$, i.e. expensive)
 - QDA has quadratic decision boundary
- LDA outperforms, if Bayes is linear
- QDA outperforms, if Bayes is quadratic

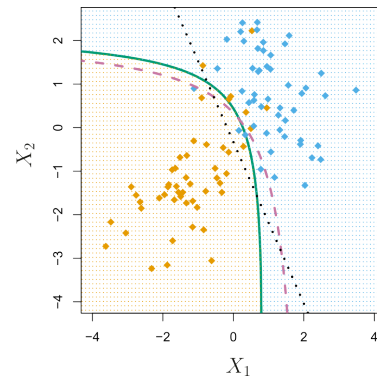
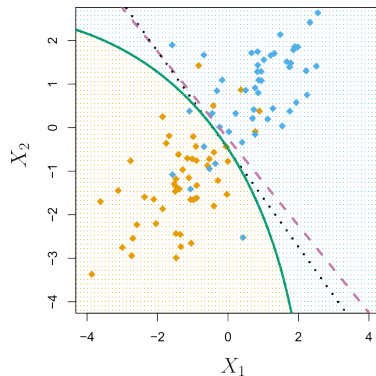


Image source: ISLP Fig. 4.9

LDA vs Logistic Regression

- Both have log odds as linear functions of x
- LDA for $k = 2$: $\log \left(\frac{p_1(x)}{p_2(x)} \right) = c_0 + c_1 x$
 - c_i are estimated MLEs of parameter of Gaussian PDF of $X|Y$
- Logistic regression for $k = 2$: $\log \left(\frac{p_1(x)}{p_2(x)} \right) = \beta_0 + \beta_1 x$
 - β_i are estimated via MLE of regression parameters
- If Gaussian assumption is correct, LDA outperforms

LDA vs KNN

- LDA is parametric
 - Requires estimation of $N_p(\mu_k, \Sigma_k)$
- KNN is non-parametric
 - No parameters to fit
 - No assumptions about the shape of the decision boundary
 - KNN outperforms with non-linear Bayes decision boundary
 - Cannot identify important variables

Summary: Choosing a Classification Method

Method	Decision Boundary	Best When	Limitations
Logistic Regression	Linear	Classes are linearly separable	Cannot capture non-linear patterns
LDA	Linear	Gaussian features, shared covariance	Same as logistic, + Gaussian assumption
QDA	Quadratic	Gaussian features, different covariances	Many parameters, needs more data
KNN	Non-linear	Complex boundaries, enough data	No feature importance, slow prediction

Lecture Summary

- **Classification** predicts categorical (qualitative) responses
 - Unlike regression which predicts quantitative values
- **Logistic Regression**
 - Models probability via sigmoid function: $p(X) = \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}}$
 - For $K > 2$ classes: OVA, OVO, or softmax (multinomial) approaches
- **Linear Discriminant Analysis (LDA)**
 - Bayesian approach: models $P(X|Y)$ assuming Gaussian distribution
 - Assumes shared covariance Σ across all classes $\rightarrow$ linear decision boundary
- **Quadratic Discriminant Analysis (QDA)**
 - Allows class-specific covariance $\Sigma_k \rightarrow$ quadratic decision boundary
 - More flexible but requires more parameters

END